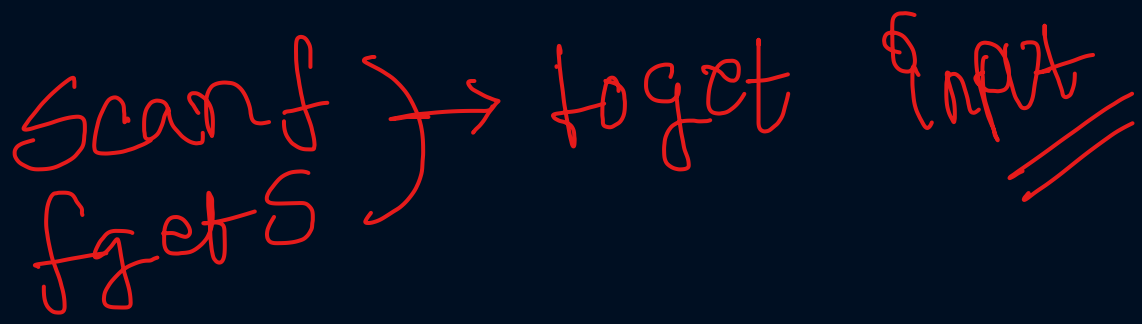
File Input Output

- Introduction to file
- Files and streams
- Sequential access files
- Reading data from file



Introduction to File

- File handling in C is the process in which we create, open, read, write, and close operations on a file.
- C language provides different functions such as fopen(), fwrite(), fread(), fseek(), fprintf(), etc. to perform input, output, and many different C file operations in our program.

Create
open
read
write
close

Why do we need File Handling in C?

- So far the operations using the C program are done on a prompt/terminal which is not stored anywhere.
- The output is deleted when the program is closed. But in the software industry, most programs are written to store the information fetched from the program.
- The use of file handling is exactly what the situation calls for.

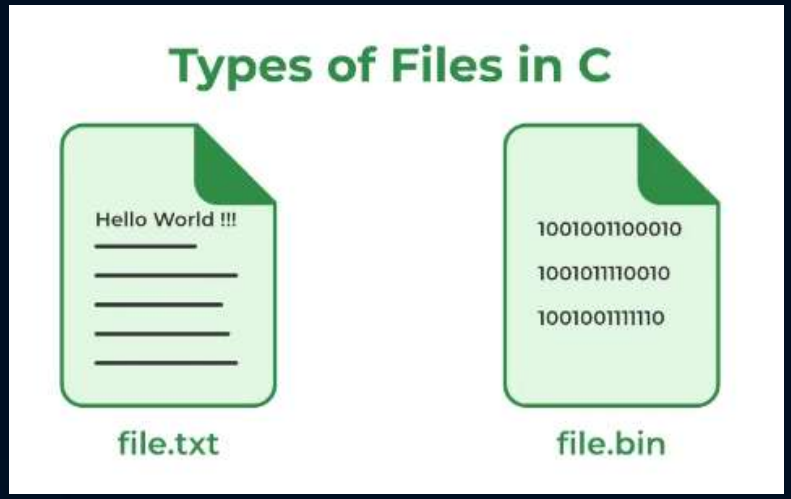
Benefits of using files

- Reusability: The data stored in the file can be accessed, updated, and deleted anywhere and anytime providing high reusability.
- Portability: Without losing any data, files can be transferred to another in the computer system. The risk of flawed coding is minimized with this feature.
- Efficient: A large amount of input may be required for some programs. File handling allows you to easily access a part of a file using few instructions which saves a lot of time and reduces the chance of errors.
- Storage Capacity: Files allow you to store a large amount of data without having to worry about storing everything simultaneously in a program.

Types of Files in C

- A file can be classified into two types based on the way the file stores the data. They are as follows:
  - Text Files
  - Binary Files

Text files
Binary files



Cont..

1. Text Files

- A text file contains data in the form of ASCII characters and is generally used to store a stream of characters.
- Each line in a text file ends with a new line character (‘\n’).
- It can be read or written by any text editor.
- They are generally stored with .txt file extension.
- Text files can also be used to store the source code.

Cont..

2. Binary Files

- A binary file contains data in binary form (i.e. 0's and 1's) instead of ASCII characters. They contain data that is stored in a similar manner to how it is stored in the main memory.
- The binary files can be created only from within a program and their contents can only be read by a program.
- More secure as they are not easily readable.
- They are generally stored with .bin file extension.

C File Operations

No	Decription	File Operation
1	Creating a new file	**fopen()** with attributes as "a" or "a+" or "w" or "w+" etc
2	Opening an existing file	**fopen()**
3	Reading from file	**fscanf()** or **fgets()**
4	Writing to a file	**fprintf()** or **fputs()**
5	Moving to a specific location in a file	**fseek(), rewind()**
6	Closing a file	**fclose()**

File Handling

In C, you can create, open, read, and write to files by declaring a pointer of type FILE

```
FILE *fptr;
fptr = fopen(filename, mode);
```

FILE is basically a data type, and we need to create a pointer variable to work with it (fptr).

Open/Create file

To actually open/create a file, use the fopen() function, which takes two parameters (*filename* and *mode*):

Parameter	Description
filename	The name of the actual file you want to open (or create), like `filename.txt`
mode	A single character, which represents what you want to do with the file (read, write or append): `w` - Writes to a file `a` - Appends new data to a file `r` - Reads from a file

Cont.. File mode

r	Open for reading.	If the file does not exist, fopen() returns NULL.
rb	Open for reading in binary mode.	If the file does not exist, fopen() returns NULL.
w	Open for writing.	If the file exists, its contents are overwritten. If the file does not exist, it will be created.
wb	Open for writing in binary mode.	If the file exists, its contents are overwritten. If the file does not exist, it will be created.
a	Open for append. Data is added to the end of the file.	If the file does not exist, it will be created.
ab	Open for append in binary mode. Data is added to the end of the file.	If the file does not exist, it will be created.

Cont.. File mode

r+	Open for both reading and writing.	If the file does not exist, fopen() returns NULL.
rb+	Open for both reading and writing in binary mode.	If the file does not exist, fopen() returns NULL.
w+	Open for both reading and writing.	If the file exists, its contents are overwritten. If the file does not exist, it will be created.
wb+	Open for both reading and writing in binary mode.	If the file exists, its contents are overwritten. If the file does not exist, it will be created.
a+	Open for both reading and appending.	If the file does not exist, it will be created.
ab+	Open for both reading and appending in binary mode.	If the file does not exist, it will be created.

Example

To actually open/create a file, use the fopen() function, which takes two parameters (*filename* and *mode*):

```
FILE *fptr;

// Create a file
fptr = fopen("filename.txt", "w");

// Close the file
fclose(fptr);
```

- you can use the w mode inside the fopen() function.

- The w mode is used to write to a file.
- However, if the file does not exist, it will create one for you.

```
fptr = fopen("C:\\directoryname\\filename.txt", "w");
```

If you want to create the file in a specific folder, just provide an absolute path (use double backslashes to create a single backslash (\\)

Write To Files

- The w mode means that the file is opened for writing.
- To insert content to it, you can use the fprintf() function and add the pointer variable (fptr in our example) and some text:

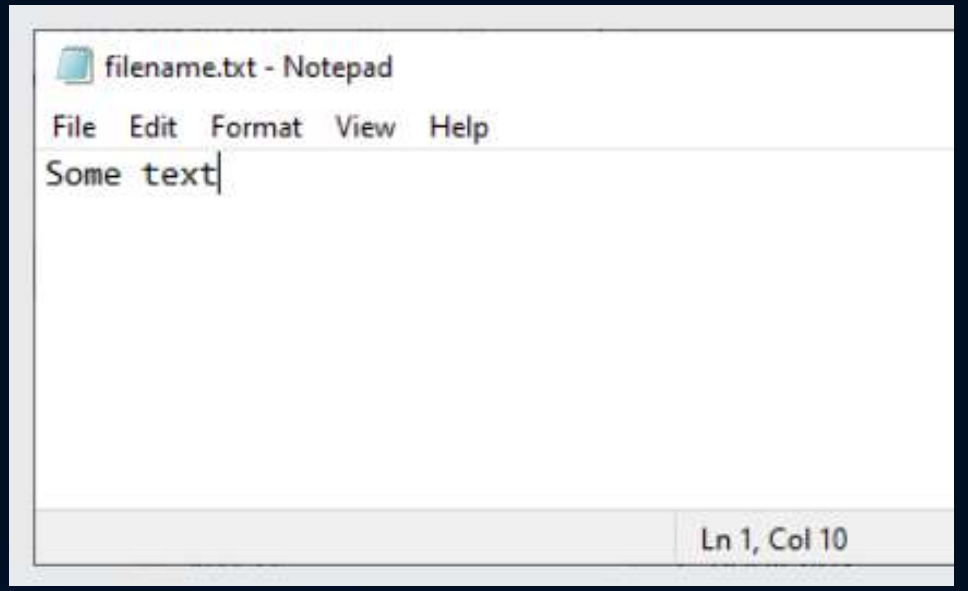
```
FILE *fptr;

// Open a file in writing mode
fptr = fopen("filename.txt", "w");

// Write some text to the file
fprintf(fptr, "Some text");

// Close the file
fclose(fptr);
```

Output will be print in *filename.txt*



Example

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int num;
   FILE *fptr;

  // use appropriate location

  fptr = fopen("D:\\testFile.txt","w");

   if(fptr == NULL)
   {
      printf("Error!");
      exit(1);
   }
   printf("Enter num: ");
   scanf("%d",&num);
   fprintf(fptr,"%d",num);
   fclose(fptr);
   return 0;
}
```

Reading From a File

- To read from a file, you can use the r mode:

```
FILE *fptr;

// Open a file in read mode
fptr = fopen("filename.txt", "r");
```

This will make the *filename.txt* opened for reading.

Example 1

Read an integer value from file

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int num;
    FILE *fptr;

    if ((fptr = fopen("C:\\program.txt","r")) == NULL){
        printf("Error! opening file");

        // Program exits if the file pointer returns NULL.
        exit(1);
    }

    fscanf(fptr,"%d", &num);

    printf("Value of n=%d", num);
    fclose(fptr);

    return 0;
}
```

Note : If the open file fail, replace "Null" with 1
If it reads one integer (%d) successfully, it returns 1. otherwise it returns a value other than 1.

Example

```
int main() {
    int celcius;
    double fahrenheit;

    FILE *fptr = fopen("D:\\fdata.txt", "r");
    if (fptr == NULL) {
        perror("Error opening file");
        return 1;
    }

    printf("Celcius        Fahrenheit\n");
    printf("=======        ==========\n");
    while (fscanf(fptr, "%d", &celcius) == 1) {
        fahrenheit = 1.8 * celcius + 32.0;
        printf("  %d\t\t%6.2f\n", celcius,fahrenheit);
    }

    fclose(fptr);
    return 0;
}
```

Example 2

Read a character in file and display it to console

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    FILE* ptr;
    char ch;

    // Opening file in reading mode
    ptr = fopen("test.txt", "r");

    if (NULL == ptr) {
        printf("file can't be opened \n");
    }

    printf("content of this file are \n");

    // Printing what is written in file
    // character by character using loop.
    do {
        ch = fgetc(ptr);
        printf("%c", ch);

} while (ch != EOF); // read until character is not EOF.

    // Closing the file
    fclose(ptr);
    return 0;
}
```

Read a string from File

- Use the fgets() function.
- The fgets() function takes three parameters:

```
fgets(myString, 100, fptr);
```

- The first parameter specifies where to store the file content, which will be in the myString array we just created.
- The second parameter specifies the maximum size of data to read, which should match the size of myString (100).
- The third parameter requires a file pointer that is used to read the file (fptr in our example).

Example

```c
#include <stdio.h>

int main() {
  FILE *fptr;
  // Open a file in read mode
  fptr = fopen("filename.txt", "r");
  char myString[100]; // Store the content of the file
  // Read the content and store it inside myString
  fgets(myString, 100, fptr);
  printf("%s", myString); // Print file content
  fclose(fptr); // Close the file
  return 0;
}
```

Thank You